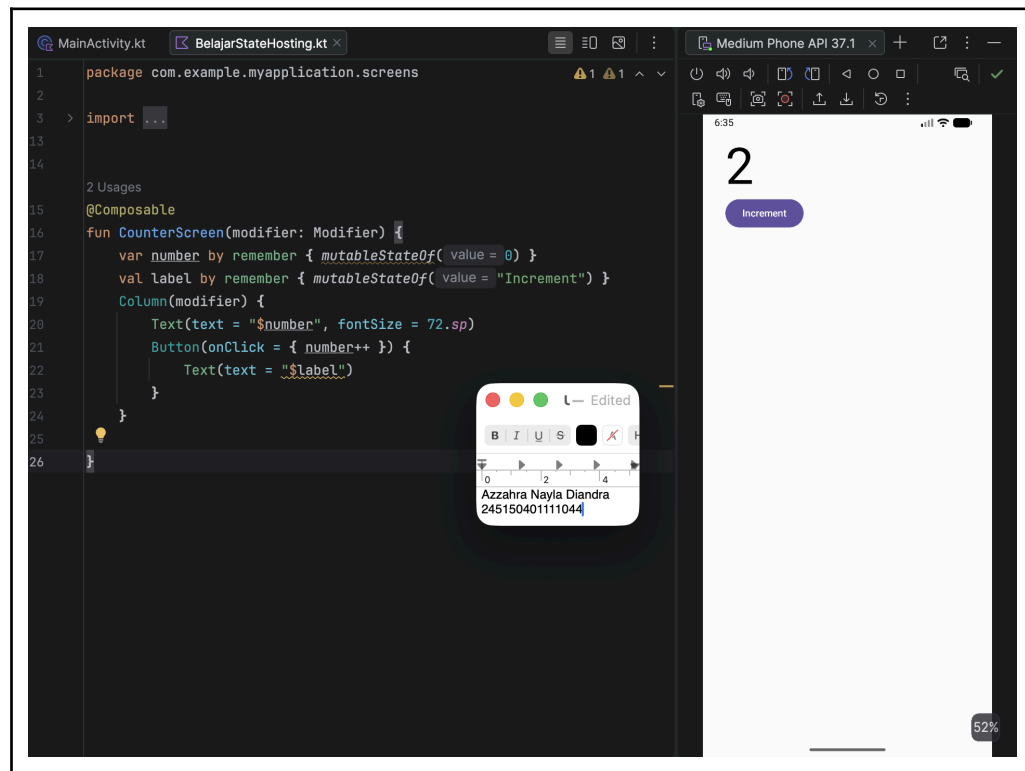


NAMA : AZZAHRA NAYLA DIANDRA
NIM : 245150401111044
KELAS : SI - C
BAB : IV – STATE HOISTING, PERUBAHAN KONFIGURASI, & SIDE-EFFECT

A. State Hoisting

1. Screenshot jika tombol “increment” diklik



- Setiap kali tombol Increment diklik, angka yang ditampilkan pada layar bertambah 1. Misalnya nilai awal 0, setelah diklik menjadi 1, kemudian 2, 3, dan seterusnya.
- Hal tersebut terjadi karena terdapat event onClick pada Button yang menjalankan:

```
Button(onClick = { number++ })
```

Setiap tombol diklik, nilai state number bertambah 1. Perubahan state tersebut menyebabkan Compose melakukan recomposition sehingga angka pada Text ikut diperbarui.

- Variabel state yang digunakan adalah:

```
var number by remember { mutableStateOf(0) }
```

Variabel number digunakan pada:

```
Text(text = "$number")
```

Jadi, perubahan nilai angka yang menyebabkan tampilan Text ikut berubah.

- d. Variabel yang digunakan adalah:

```
val label by remember { mutableStateOf("Increment") }
```

Kemudian digunakan pada:

```
Text(text = "$label")
```

Jadi, label menjadi sumber nilai untuk teks yang ditampilkan pada Button.

2. Percobaan kedua kode diubah agar bertambah 2 setiap diklik.

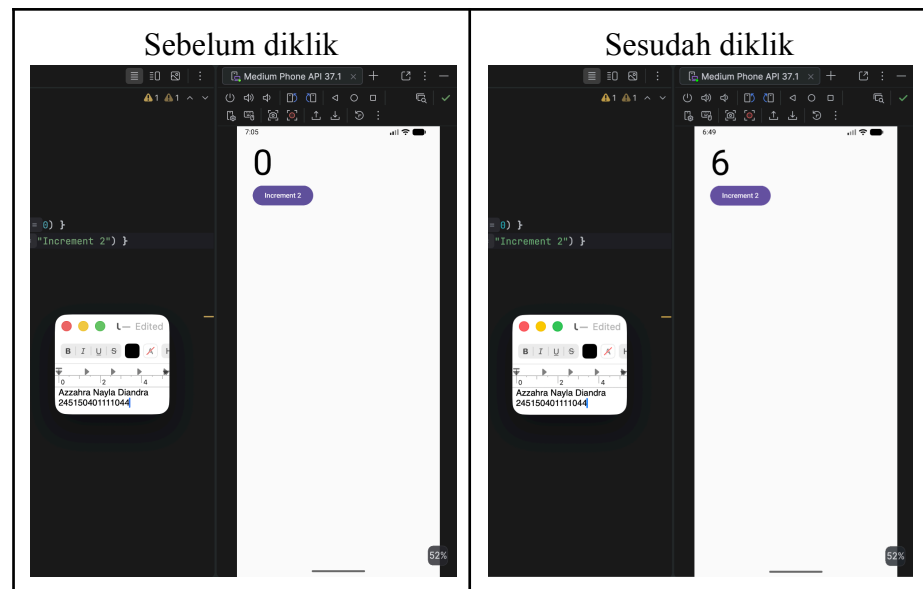
- a. Agar angka bertambah 2 setiap kali tombol diklik, bagian:

```
Button(onClick = { number++ }) diubah menjadi Button(onClick = {  
number += 2 })
```

Kemudian label "Increment" diubah menjadi "Increment 2"

Perubahan dilakukan di dalam Composable CounterScreen().

- b. Screenshot setelah kode diubah dan diklik "Increment 2"



3. (No.4) Melakukan state dan event hoisting

- a. Ya, program tetap berjalan dengan baik dan menghasilkan tampilan yang sama. Perbedaannya terletak pada lokasi pengelolaan state dan event, bukan pada hasil tampilan aplikasi.
- b. Proses increment dilakukan pada lambda onClick ketika CounterScreen dipanggil:

```
onClick = { number++ }
```

- c. State number dideklarasikan di dalam onCreate():

```
var number by remember { mutableStateOf(0) }
```

Sedangkan label Button diberikan melalui parameter:

```
label = "Increment"
```

Dengan demikian, CounterScreen() hanya menerima data dan event dari luar.

4. (No.5) Mengubah kode program sesuai perintah

- a. Mengubah label = "Increment" menjadi label = "Double"
- b. Mengubah nilai awal 0 → remember { mutableStateOf(0) } menjadi 1 → remember { mutableStateOf(1) }
- c. Mengubah fungsi onClick = { number++ } menjadi onClick = { number *= 2 } agar number digandakan
- d. Menulis kode state dan onCreate()

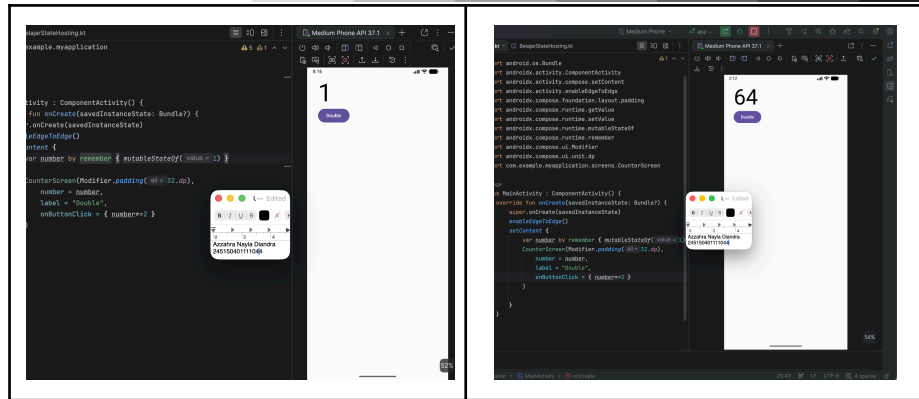
```
setContent {
    var number by rememberSaveable { mutableStateOf(1) }

    CounterScreen(modifier = Modifier.padding(32.dp),
        number = number,
        label = "Double",
        onClick = { number *= 2 }
    )
}
```

5. (No.6) Jalankan program

- a. Screenshot jika tombol "Double" diklik

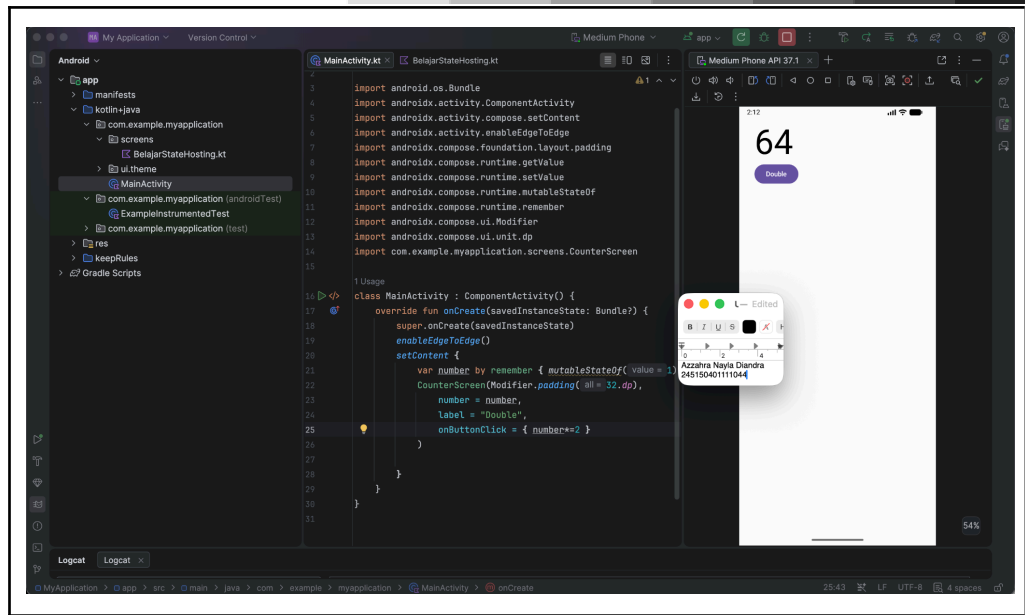
Sebelum diklik	Sesudah diklik
----------------	----------------



- b. Perubahan dilakukan pada fungsi onCreate(), terutama pada deklarasi state dan lambda onClick.
 - c. Tidak ada perubahan pada logika internal CounterScreen(). Composable tersebut tetap hanya menerima number, label, dan onClick sebagai parameter.
6. (No.7) State hoisting membuat perubahan kebutuhan aplikasi lebih mudah dilakukan karena logika state dan event berada di luar Composable UI. Misalnya ketika kebutuhan berubah dari tombol Increment menjadi Double, kita cukup mengubah nilai label dan logika onClick pada pemanggilnya tanpa mengubah struktur CounterScreen(). Hal ini membuat Composable menjadi lebih reusable, mudah dipelihara, dan lebih mudah diuji. State hoisting juga menerapkan prinsip Single Source of Truth, karena state hanya dikelola di satu tempat.

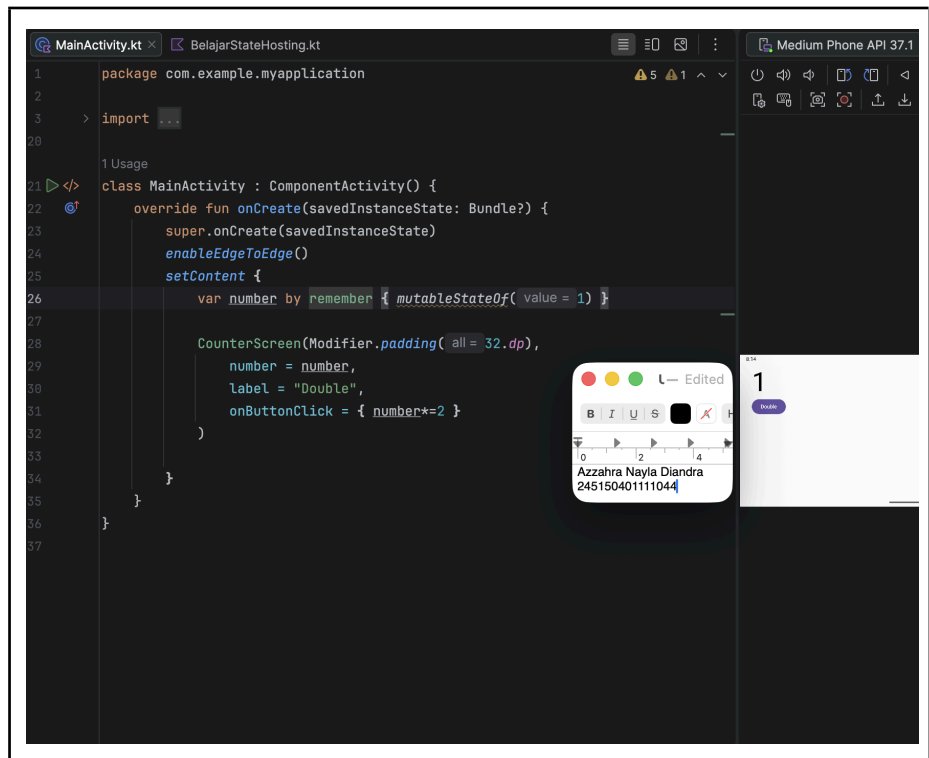
B. Perubahan Konfigurasi

1. Screenshot saat tombol “Double” diklik



2. Ubah konfigurasi posisi portrait menjadi landscape

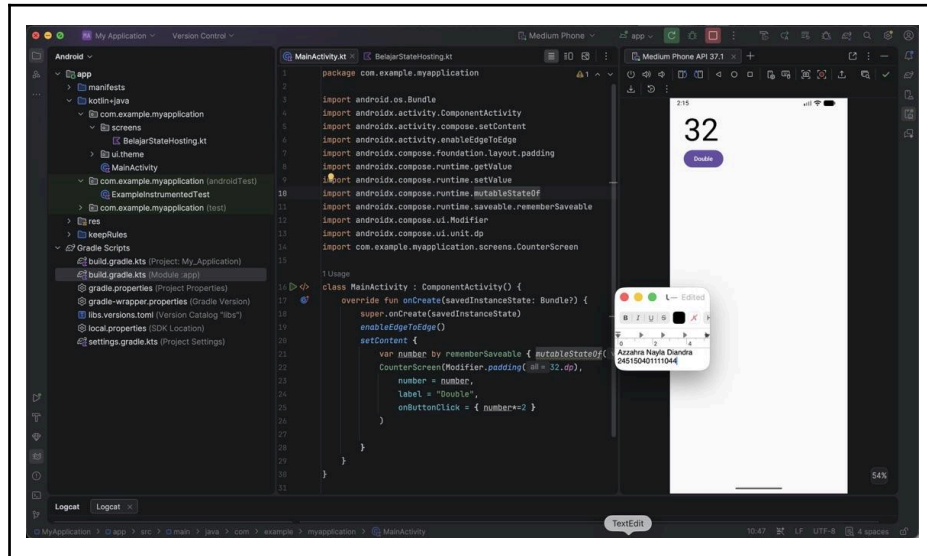
a. Screenshot



- b. Setelah perangkat diputar ke posisi landscape, Activity mengalami perubahan konfigurasi dan dibuat kembali oleh Android. Angka yang ditampilkan kembali menjadi 1. Hal ini terjadi karena state

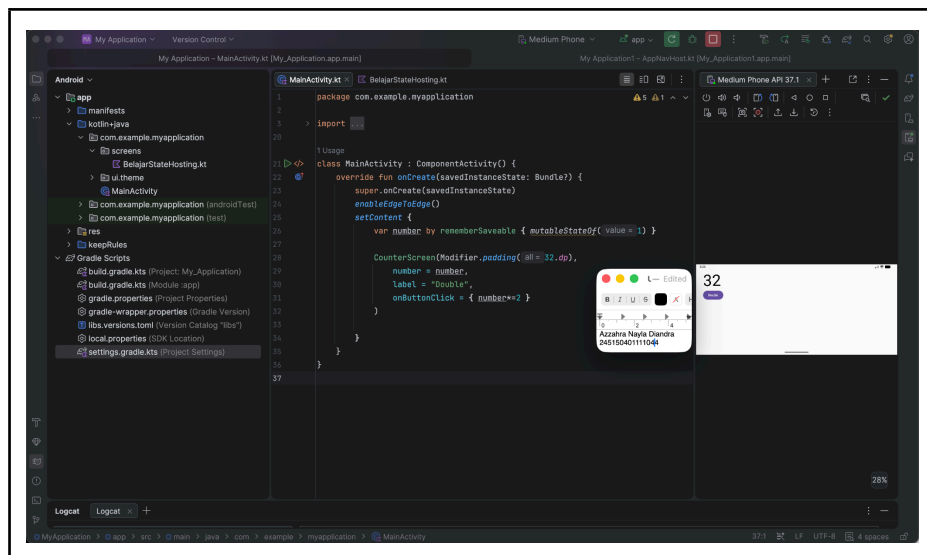
menggunakan `remember { mutableStateOf(1) }` yang dapat mempertahankan state selama recomposition, tetapi tidak mempertahankannya ketika Activity dibuat ulang akibat perubahan konfigurasi seperti rotasi layar. Akibatnya, nilai number kembali ke nilai awal 1.

c. Screenshot setelah mengubah kode state `rememberSaveable`



3. Setelah mengubah kode state jadi `rememberSaveable { mutableStateOf(1) }`

a. Screenshot setelah memutar posisi menjadi landscape



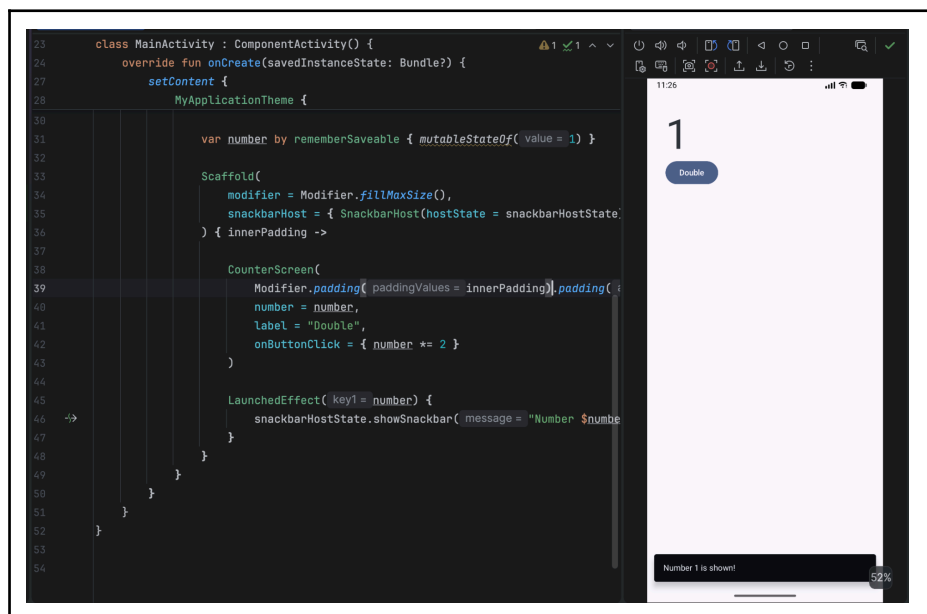
- b. Setelah rotasi dengan rememberSaveable, nilai tetap 32. Karena rememberSaveable menyimpan nilai state sehingga dapat dipulihkan kembali ketika terjadi perubahan konfigurasi.
 - c. remember mempertahankan nilai selama proses recomposition, tetapi state dapat hilang ketika Activity dihancurkan dan dibuat kembali, misalnya saat rotasi. Sedangkan rememberSaveable dapat mempertahankan nilai tersebut melewati perubahan konfigurasi dengan menyimpannya dalam saved instance state/Bundle.
4. Contoh kasus nyata penggunaan rememberSaveable adalah form pendaftaran akun. Jika pengguna sudah mengetik nama, email, atau alamat kemudian memutar layar, data yang sudah dimasukkan tetap dapat ditampilkan. Contoh:
- ```
var name by rememberSaveable { mutableStateOf("") }
```
- Manfaatnya, pengguna tidak perlu mengisi ulang data setelah terjadi perubahan konfigurasi sehingga pengalaman penggunaan aplikasi menjadi lebih nyaman.

## C. Launched Effect

### Percobaan 1

1. Jalankan program
  - a. Saat aplikasi pertama kali dibuka, nilai awal number adalah 1, sehingga akan muncul Snackbar dengan pesan:

**"Number 1 is shown!"**



- b. SnackBar muncul tanpa pengguna menekan tombol karena `LaunchedEffect(number)` langsung dijalankan ketika `Composable` pertama kali masuk ke `composition`. Pada saat itu nilai `number` sudah memiliki nilai awal sehingga effect langsung berjalan
- c. SnackBar dipicu oleh perubahan atau masuknya nilai `number` ke dalam `LaunchedEffect`. Bagian kodenya adalah:

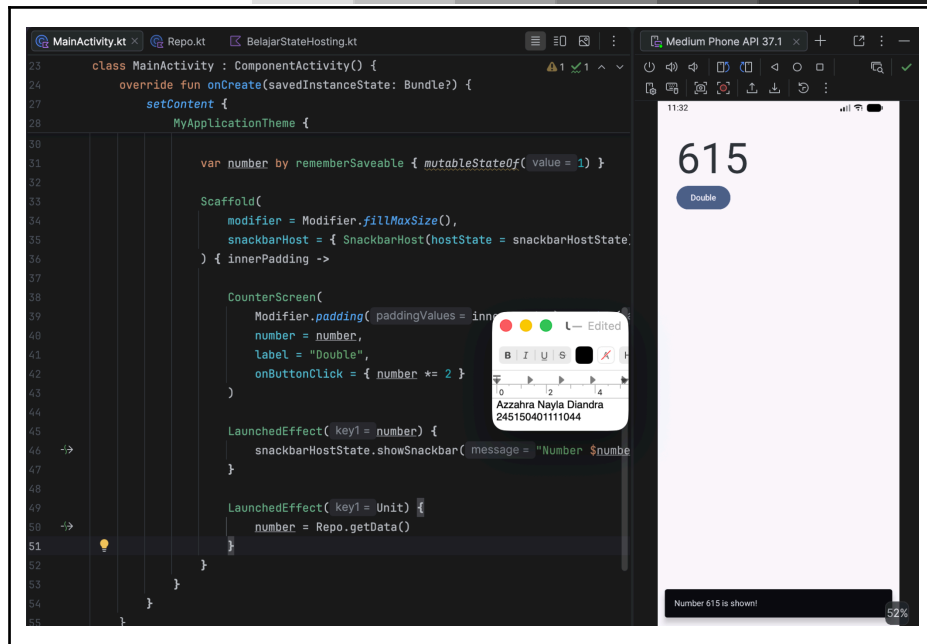
```
LaunchedEffect(number) {
 snackbarHostState.showSnackbar("Number $number is shown!")
}
```

Ketika tombol `Double` ditekan dan `number` berubah, `LaunchedEffect(number)` dijalankan kembali dengan nilai terbaru.

- 2. Setelah menekan tombol “Double”
  - a. Jika nilai awal `number` = 1, setelah tombol ditekan nilainya menjadi 1 → 2. Setelah nilai berubah menjadi 2, SnackBar akan menampilkan:  
**"Number 2 is shown!"**
  - b. Karena tombol `Double` mengubah state `number`. Perubahan state tersebut menjadi pemicu `LaunchedEffect(number)` untuk menjalankan kembali blok effect dengan nilai `number` yang baru.  
Alurnya klik `Double` → `number` berubah → `recomposition` → `LaunchedEffect(number)` dijalankan → SnackBar muncul.

## Percobaan 2

- 1. Jalankan program
  - a. Pada awalnya, angka masih menggunakan nilai awal, yaitu 1, dan muncul SnackBar “Number 1 is shown!”. Setelah proses pengambilan data selesai, angka berubah menjadi angka acak hasil dari `Repo.getData()`. Angka pastinya dapat berbeda setiap kali program dijalankan.
  - b. Setelah sekitar dua detik, angka berubah menjadi angka acak hasil dari `Repo.getData()`. Angka pastinya dapat berbeda setiap kali program dijalankan.



c. Perubahan angka dipicu oleh:

```
LaunchedEffect(Unit) {
 number = Repo.getData()
}
```

Setelah `Repo.getData()` selesai, nilai `number` diperbarui menjadi angka acak antara 100 sampai 999 yang dihasilkan oleh `Random.nextInt(100, 1000)`. Perubahan `number` kemudian menyebabkan `LaunchedEffect(number)` menjalankan kembali `SnackBar` dengan nilai terbaru.

## 2. Percobaan side-effect dan LaunchedEffect

- a. **LaunchedEffect(state)** bergantung pada nilai state tertentu. Effect dijalankan ketika Composable pertama kali masuk composition dan akan dijalankan kembali ketika nilai state berubah. Sedangkan **LaunchedEffect(Unit)** menggunakan Unit sebagai key sehingga effect dijalankan ketika Composable pertama kali masuk composition dan tidak dijalankan ulang hanya karena state lain berubah. Berikut adalah perbedaan kodenya:

|                                               |                                                                 |
|-----------------------------------------------|-----------------------------------------------------------------|
| <pre>LaunchedEffect(number) {     ... }</pre> | <pre>LaunchedEffect(Unit) {     number = Repo.getData() }</pre> |
|-----------------------------------------------|-----------------------------------------------------------------|

b. Kapan dijalankan?

| Konstruk               | Kapan dijalankan?                                                          |
|------------------------|----------------------------------------------------------------------------|
| LaunchedEffect(number) | Saat pertama masuk composition dan setiap kali nilai number berubah        |
| LaunchedEffect(Unit)   | Saat pertama masuk composition, tidak diulang karena perubahan state biasa |

Jadi, LaunchedEffect(number) cocok digunakan ketika suatu proses harus merespons perubahan state tertentu, sedangkan LaunchedEffect(Unit) cocok untuk proses yang ingin dijalankan sekali ketika Composable pertama kali muncul, misalnya mengambil data awal dari repository/API.